



Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Platform Oyunu Mekaniği

PyGame ile Oyun Geliştirme – Hafta 11

Dr. Tolga BERBER

Bu Hafta Ne Öğreneceğiz?

- 1 Gelişmiş Karakter Hareketi
- 2 Platform Çarpışmaları
- 3 Düşman Davranışları
- 4 Oyun Sistemleri

Gelişmiş Karakter Hareketi

- Ham hız yerine **ivme** (acceleration)
- Tuş bırakınca **sürtünme** ile yavaşlama
- **MAX_HIZ** ile maksimum koşu hızı
- **ZIP_HIZI** ve **YERCEKIMI**: zıplama profili

3 Sütun

Sıkılık
Ağırlık
Affediş

İvme + Sürtünme ile Hareket

```
IVME = 0.7
MAX_HIZ = 5.0
SURTUNME = 0.4

if tuslar[K_LEFT]:    self.vx -= IVME
elif tuslar[K_RIGHT]: self.vx += IVME
else:
    if self.vx > 0:    self.vx = max(0, self.vx - SURTUNME)
    elif self.vx < 0: self.vx = min(0, self.vx + SURTUNME)

self.vx = max(-MAX_HIZ, min(MAX_HIZ, self.vx))
```

Tek Parametre Değiştir

IVME, SURTUNME, MAX_HIZ tek tek test et.

- Platformdan ayrıldıktan sonra kısa bir süre zıplayabilme
- Tipik değer: 6 kare (≈ 100 ms)
- Yerde $\rightarrow$ havada geçişinde başlatılır
- Wile E. Coyote: uçurumdan sonra havada asılı kalma

Değer Rehberi

0 = katı

6 = standart

20+ = sihirli

```
COYOTE_SURE = 6

# Her kare:
eski_yerde = self.yerde
# ... carpisma kontrolu ...
if eski_yerde and not self.yerde:
    self.coyote_sayac = COYOTE_SURE
elif self.coyote_sayac > 0:
    self.coyote_sayac -= 1

def zipla(self):
    if self.yerde or self.coyote_sayac > 0:
        self.vy = -11
        self.coyote_sayac = 0 # tüketildi
```

Jump Buffer

```
BUFFER_SURE = 6

# KEYDOWN olayında:
def zipla_istegi(self):
    self.buffer_sayac = BUFFER_SURE

# Her kare:
if self.buffer_sayac > 0:
    self.buffer_sayac -= 1
if self.buffer_sayac > 0 and (self.yerde or self.coyote_sayac > 0):
    self.vy = -11
    self.buffer_sayac = 0
```

Fark

Coyote: geçmiş affedişi. Buffer: gelecek affedişi.

Variable + Double Jump

```
# Variable jump: KEYUP olayinda yukari hizi kes
def tus_birakildi(self):
    if self.zipliyor and self.vy < 0:
        self.vy *= 0.4

# Double jump: kalan_ziplama sayaciyla
if self.kalan_ziplama > 0:
    self.vy = -11 if self.yerde else -9
    self.kalan_ziplama -= 1

# yere deyince yenile:
if self.yerde:
    self.kalan_ziplama = MAX_ZIPLAMA
```

Platform Çarpışmaları

4 Yön + 4 Düzeltme

- **Alt** ($v_y > 0$): $\text{bottom} = \text{top}$
- **Üst** ($v_y < 0$): $\text{top} = \text{bottom}$
- **Sağ** ($v_x > 0$): $\text{right} = \text{left}$
- **Sol** ($v_x < 0$): $\text{left} = \text{right}$

Sözlük Döndür

```
temas = {  
    "alt": True,  
    "ust": False,  
    "sol": False,  
    "sag": False,  
}
```

```
def _dikey_tek_yonlu(self, platformlar, eski_alt):  
    for p in platformlar:  
        if self.rect.colliderect(p):  
            # Dususte VE onceki kare ustteyse  
            if self.vy > 0 and eski_alt <= p.top + 1:  
                self.rect.bottom = p.top  
                self.vy = 0  
                self.yerde = True
```

Drop-Through

Aşağı tuşu + SPACE = 10 kare platform yok sayılır.

Hareketli Platform - Delta Transferi

```
# Platform:
def guncelle(self):
    eski = Vector2(self.rect.topleft)
    # ... hareket ...
    self.delta = Vector2(self.rect.topleft) - eski

# Oyuncu (her kare basi):
if self.uzerindeki_platform:
    self.rect.x += round(platform.delta.x)
    self.rect.y += round(platform.delta.y)
```

Altın Kural

Delta'yı klavye girdisinden **once** uygula.

Crumbling + Bouncy Platformlar

Crumbling (Dağılan)

FSM: sağlam $\rightarrow$ sarsiliyor (30 kare) $\rightarrow$ yok (120 kare) $\rightarrow$ sağlam

Bouncy (Yaylı)

Üstten iniş: $v_y = -18$
Zıplama haklarını yenile

Düşman Davranışları

FSM (Finite State Machine)

Durumlar

- **idle**: bekleme
- **patrol**: iki nokta arası gezi
- **chase**: oyuncuyu takip
- **attack**: saldırı/ateş

Her Kare

```
durum_metotlari[self.durum]()  
Koşullara göre self.durum = "chase"
```



```
def _ucurum_var_mi(self, katilar):
    test_x = self.rect.right + 2 if self.vx > 0 else self.rect.left
        - 6
    test = pygame.Rect(test_x, self.rect.bottom + 2, 4, 6)
    for k in katilar:
        if test.collidect(k):
            return False # ayak altI dolu
    return True # ucurum!

def guncelle(self, oyuncu, katilar):
    if self._ucurum_var_mi(katilar):
        self.vx *= -1
    self._fizik(katilar)
```

```
FARK_MESAFE = 200
```

```
def _oyuncu_yakin_mi(self, oyuncu):  
    a = pygame.Vector2(self.rect.center)  
    b = pygame.Vector2(oyuncu.rect.center)  
    return a.distance_to(b) <= FARK_MESAFE  
  
# FSM:  
if self.durum == "patrol" and self._oyuncu_yakin_mi(oyuncu):  
    self.durum = "chase"  
elif self.durum == "chase" and not self._oyuncu_yakin_mi(oyuncu):  
    self.durum = "patrol"
```

Ranged AI - Mermi Fırlatma

```
class Mermi(pygame.sprite.Sprite):
    def update(self, katilar):
        self.rect.x += int(self.vx)
        self.rect.y += int(self.vy)
        for k in katilar:
            if self.rect.colliderect(k):
                self.kill()

# Ates etme:
baslangic = Vector2(self.rect.center)
hedef = Vector2(oyuncu.rect.center)
yon = (hedef - baslangic).normalize()
Mermi(bx, by, yon.x * 4, yon.y * 4)
```

Nişan İpucu

15 kare önce kırmızı çizgi göster!

```
for d in list(dusmanlar):
    if oyuncu.rect.colliderect(d.rect):
        if (oyuncu.vy > 0 and
            oyuncu.rect.bottom < d.rect.top + 16 and
            d.oldurulebilir):
            d.kill()
            oyuncu.vy = -9 # kucuk ziplama
        else:
            oyuncu.hasar_al(1, kaynak_rect=d.rect)
```

Üç Koşul Birlikte

$vy > 0$ (düşüşte) + alt kenar üstte + oldurulebilir

Oyun Sistemleri

```
# Oyuncu'da:
self.efektler = {} # {"hiz": 600, "kalkan": 900}

def power_up_al(self, tur):
    sureler = {"hiz": 600, "zipla": 600, "kalkan": 900}
    self.efektler[tur] = sureler[tur]

# Her kare:
for tur in list(self.efektler.keys()):
    self.efektler[tur] -= 1
    if self.efektler[tur] <= 0:
        del self.efektler[tur]
```

Yenileme

- Kalp sprite ile +1 can
- Pasif: 5 saniyede +1
- $\min(\text{max_can}, \text{can} + n)$

Hasar

- Ölçekli: düşman türüne göre
- Knockback: geriye fırlat
- Spawn'a döndürme

```
I_FRAME_SURE = 60    # 1 saniye

def hasar_al(self, miktar):
    if self.iframe_sayac > 0:
        return # hala dokunulmaz
    self.can -= miktar
    self.iframe_sayac = I_FRAME_SURE

def ciz(self, ekran):
    # On/Off flicker: her 4 karede bir gizle
    if self.iframe_sayac > 0 and (self.iframe_sayac // 4) % 2 == 0:
        return
    ekran.blit(self.image, self.rect)
```


Dosya Bölümü

- `main.py` – ana döngü + Seviye
- `entities.py` – Oyuncu, Dusman{Patrol, Chase, Ranged}, PowerUp, Kalp
- `platforms.py` – HareketliPlatform, CrumblingPlatform, BouncyPlatform

Tile ID Şeması

1=boş 2=solid 3=spawn 4=altın 5=diken 6=one-way 7=hareketli
8=crumbling 9=bouncy

Bu Hafta Öğrendiklerimiz

- Game feel: ivme, sürtünme
- Coyote + jump buffer
- Variable + double jump
- One-way, moving, crumbling, bouncy
- FSM ile patrol/chase/ranged AI
- Power-up + can + i-frames
- Modüler mimari

Gelecek Hafta

- State machine tasarımı
- JSON dosya işlemleri
- Save/Load sistemi
- Ayarlar ve konfigürasyon